FFFA Donor Management: Prompt Guide

28 October 2026 · for setting up, running and changing the system with an AI coding assistant

FFFA doesn't need a programmer to keep this system going. An AI coding assistant (e.g. Claude Code, GitHub Copilot in VS Code, or Cursor) can install, run, change and deploy it, if it gets the right context and clear instructions, and if a person checks and approves its work. This guide gives you all three.

---

1. Approved context: what to give the assistant

Always start a session in the project folder with this message. It points the assistant at the project's own rules, so it builds the way the system was designed.

> You are helping Free Food For All (FFFA), a Singapore charity, maintain its donor management system. Before doing anything, read README.md, docs/SETUP_GUIDE.md and docs/SYSTEM_DESIGN_v1.3.md, and follow the conventions in README.md. Explain each step in plain English before you do it. Never put real donor data, passwords or API keys in code, commits or chat. Ask me before you delete anything, push code or change anything online.

Give the assistant	Never give the assistant
The project folder and the documents in `docs/`	Real donor names, emails, mobiles or giving records
Error messages, with keys and personal details removed	Passwords, API keys, the `.env` file or database connection strings
Screenshots of the app with practice data	Exports of the donor list
A sample import file with made-up rows	NRIC, FIN, UEN or bank details

To test with realistic data, use the practice donors (`npm run demo`) or the practice Excel files (`npm run demo:files`).

---

2. Prompt templates: getting started

Copy, paste, and change the parts in `<angle brackets>` .

Get the code

> Clone https://github.com//FFFA-Donor-Management-System into my Documents folder and open it.

Install and run

> Check that Git and Node.js 20+ are installed (tell me how to install anything missing). Then install the project with `npm install --legacy-peer-deps` , run `npm test` , and start it with `npm run demo` . Tell me the address to open in my browser.

Try the Excel import with practice files

> Run `npm run demo:reset` , then `npm run demo:files` , then start the app with `npm run dev` . Tell me which file in docs/samples/ to upload on the Import tab.

Explain something

> Explain in plain English how <the follow-up rules / the Excel import / the AI drafting> works in this project, which files are involved, and what I can safely change in the Settings screen versus in code.

Get up to date with GitHub

> Pull the latest main from GitHub, reinstall packages with `npm install --legacy-peer-deps`, and run the tests. Tell me if anything fails.

---

3. Prompt templates: going live

Do these in order, one per session. The Setup Guide (section 8) explains each service.

1. Staff login (required before real data)

> Implement `supabaseStaffResolver` in server/middleware/auth.ts using Supabase Auth with email sign-in, as described in docs/SYSTEM_DESIGN_v1.3.md. Only people in the `staff` table may sign in, and their role comes from that table. Add a simple login screen and a sign-out button. Keep `AUTH_MODE=dev` working for local practice. Add tests, run `npm test` and `npm run lint`, and explain what I need to set up in Supabase.

2. Database

> Walk me through creating a Supabase project in the Singapore region and finding the connection string. Tell me exactly where to paste it in the hosting settings, and don't ask me to paste it into this chat.

3. Hosting

> Help me deploy this app to Render as a Web Service from our GitHub repository. Build command: `npm install --legacy-peer-deps && npm run build`. Start command: `npm start`. Walk me through each screen and tell me which environment variables to add (names only, I'll enter the values myself).

4. Daily follow-up check

> Make the follow-up rules (runRulesEngine) run automatically once a day at 7am Singapore time on the server, and log each run in the activity log. Add a test.

5. Singapore dates

> The app works out "today" in UTC, so before 8am Singapore time it can be a day behind. Make all "today" calculations use Asia/Singapore, in both the server and the database queries. Add tests.

6. Add staff

> Show me how to add a staff member to the `staff` table in Supabase (email, name, role staff or admin) using the Supabase dashboard, step by step.

---

4. Prompt templates: everyday changes

Add a missing screen (the API already exists for each of these)

> Add buttons on the Follow-ups screen for Done (with a short summary), Snooze (pick a date) and Dismiss (with a reason), using the existing endpoints in server/routes/followups.ts. Match the style of the existing screens.

> On the donor profile (Donors tab), let admins edit contact details, archive a donor, void a donation with a reason, and add or complete key dates, using the existing endpoints in server/routes/donors.ts.

> Add a Giving.sg / GIVE.asia CSV upload to the Import tab using the existing /api/import/preview and /api/import/commit endpoints. Show the preview statuses before importing.

> Add an admin-only Activity screen listing the latest entries from /api/activity.

Change wording or rules

> Change the message template in server/services/ai/fallback.ts to . Update the AI instructions in server/services/ai/prompts.ts to match, and bump PROMPT_VERSION. Run the tests and show me a draft for each practice donor.

> Add a new field to donors. Use a new migration file (never edit an existing one), update the shared types, the Excel template and the donor profile screen, and add tests.

Fix a problem

> When I , I see <this error, with personal details removed>. Find the cause, explain it, fix it without deleting or weakening any tests, and run `npm test`.

---

5. Verification: check the assistant's work

Before accepting any change, ask:

> Run `npm test` and `npm run lint`, then start the app with `npm run demo` and tell me exactly what to click to see the change.

Then check it yourself:

- [ ] **Tests pass.** All of them, and none were deleted or switched off to make them pass.
- [ ] **You saw it working** in the browser, with practice data.
- [ ] **The old flow still works:** refresh follow-ups → draft → approve → mark as sent, and a major donor is still blocked from AI drafting.
- [ ] **Only the expected files changed:** *"List the files you changed and why, in one line each."*
- [ ] **No secrets or real data** in the changes: *"Check the changes for API keys, passwords, connection strings or real personal data."*
- [ ] **Database changes are a new migration file**, never an edit to an existing one.
- [ ] **Safeguards are intact:** contact details never sent to the AI, nothing sent automatically, staff approve every message, exports admin-only.

If something looks wrong: *"Undo your last change and explain what went wrong."*

6. Staff approval: from change to live

Changes go live only after a person approves them on GitHub.

1. **Branch.** *"Create a new branch called from the latest main."*
2. **Change and verify** (sections 4 and 5).
3. **Pull request.** *"Commit with a clear message, push the branch and open a pull request that explains the change in plain English, with how to test it."*
4. **Review.** The system owner (or a second person) reads the pull request on GitHub and tries the change. For anything touching donor data, login, exports or AI wording, the system owner approves.
5. **Merge.** Click *Merge pull request*. If hosting is set up, the live site updates automatically. Check it once after merging.
6. **If a live change breaks something:** on GitHub, open the merged pull request → *Revert* → merge the revert. Then fix it calmly on a new branch.

Change	Who approves
Wording, layout, a new screen using an existing API	System owner
Follow-up rules or tiers	Admin, in the **Settings** screen. No code change needed.
New fields, AI instructions, login, exports, anything touching personal data	System owner, plus FFFA's data protection contact
Hosting, billing, new services	FFFA management